

AIML Study Guide

- [AI/ML Intern Assessment — Study & Answer Guide](#)
 - [SECTION A — MCQ Answers & Why](#)
 - [SECTION B — Theory & Coding](#)
 - [Q16 — ML Pipeline \(churn prediction, imbalanced\)](#)
 - [Q17 — CNN in PyTorch](#)
 - [Q18 — RAG System](#)
 - [Q19 — Deployment \(FastAPI + Docker\)](#)
 - [Q20 — Agentic AI with n8n \(support ticket assistant\)](#)
 - [Last-minute priority if time is short](#)

AI/ML Intern Assessment — Study & Answer Guide

Use this to understand the *reasoning*, not just memorize letters — several questions test the same underlying concept in different disguises, so understanding beats memorizing.

SECTION A — MCQ Answers & Why

Q1 — Answer: A `df.groupby('city')['sales'].sum().sort_values(ascending=False)` Group by the category column, aggregate the numeric column, then sort. B/C/D have the group/aggregate order backwards or apply sum before grouping (which doesn't work as intended).

Q2 — Answer: B `WHERE` filters individual rows **before** grouping happens; `HAVING` filters **after** grouping, on aggregated values (like `COUNT(*)`). You can't write `WHERE COUNT(*) > 5` because `COUNT(*)` doesn't exist yet at the row-filtering stage — that's exactly why `HAVING` exists.

Q3 — Answer: C HTTP 422 = "Unprocessable Entity" — the JSON was syntactically valid but failed validation (wrong type, missing required field). 400 = malformed request, 404 = not found, 401/403 = auth issues.

Q4 — Answer: A Fitting `StandardScaler` on the **full dataset** before splitting lets test-set statistics (mean/std) leak into training — the model indirectly "sees" test data. Always fit preprocessing on train only, then transform test with those same parameters.

Q5 — Answer: B `StratifiedKFold` preserves the class ratio (3% positive) in every fold. Plain `KFold` or a single random split risks folds with zero or very few positive examples, making evaluation unstable/meaningless.

Q6 — Answer: B Precision and Recall (or PR-AUC) — with 99.2% of the data being "not fraud," a model that predicts "not fraud" every time scores ~99% accuracy while catching zero fraud. Precision/Recall exposes that it's not actually detecting the minority class.

Q7 — Answer: C Increasing model depth/width **least** helps — that's classic overfitting (huge train/val gap), and more capacity makes overfitting *worse*. Dropout, regularization, more data, and early stopping all directly fight overfitting.

Q8 — Answer: B K-Means assumes roughly spherical, similarly-sized clusters and **is** sensitive to feature scaling (unscaled features with large ranges dominate distance calculations). It does *not* auto-pick `k`, doesn't natively detect outliers/noise (that's DBSCAN), and can converge to different local optima depending on initialization (which is why `k-means++` / multiple inits are used).

Q9 — Answer: B `(1, 16, 16, 16) . padding=1, kernel=3, stride=1` keeps spatial size the same (32×32)

after conv, but changes channels 3→16. Then `MaxPool2d(2)` halves spatial dims: 32→16. So: (batch=1, channels=16, H=16, W=16).

Q10 — Answer: B `CrossEntropyLoss` expects **raw logits** — it internally applies log-softmax + NLL loss. If you also apply softmax in your model's final layer, you're double-applying it, which breaks training. Targets are integer class indices, not one-hot floats.

Q11 — Answer: B Self-attention has no inherent sense of sequence order (it treats tokens as an unordered set mathematically) — positional encodings inject that order information explicitly.

Q12 — Answer: B Cosine similarity between embedding vectors is the standard measure of semantic closeness in vector databases (measures angle/direction, not raw string similarity).

Q13 — Answer: B RAG wins when you need answers grounded in **frequently updated** documents with **citations** — you can update the document store without retraining anything. Fine-tuning is better for fixed output style/format or teaching genuinely new behavior/language.

Q14 — Answer: B Tool calling: the LLM does **not** execute anything itself — it returns a structured request (function name + arguments), your application code actually runs it, and the result is sent back to the model to continue reasoning. This is the single most important agentic AI concept — know it cold.

Q15 — Answer: B Webhook node (trigger) → HTTP Request node → Respond to Webhook node. Webhook = receives the external trigger; HTTP Request = calls the third-party API; "Respond to Webhook" = sends a response back to whoever called it. This is the standard n8n pattern for building an API-like automation.

SECTION B — Theory & Coding

Q16 — ML Pipeline (churn prediction, imbalanced)

(a) Full pipeline with `ColumnTransformer`:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

df = pd.read_csv("customers.csv")
X = df.drop(columns=["churn"])
y = df["churn"].map({"Yes": 1, "No": 0})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

numeric_features = ["tenure_months", "monthly_charges", "total_charges"]
categorical_features = ["contract_type", "payment_method", "internet_service"]

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(random_state=42))
])

pipeline.fit(X_train, y_train)

```

(b) Why preprocessing must live inside the Pipeline: This prevents **data leakage**. If you scale/impute on the full DataFrame before splitting, statistics from the test set (its mean, its most frequent category, etc.) influence the transformation applied to training data. The model then implicitly “knows” something about the test distribution before ever training, which **inflates your reported score** — it looks better than it will perform on truly unseen data in production.

(c) Metric choice + tuning: With only 18% churners, **accuracy is misleading**. Report **Recall, Precision, F1, and PR-AUC** — in a churn business context, missing an actual churner (false negative) usually costs more than a false alarm, so **Recall** (or F1 if you need balance) is usually prioritized.

```

from sklearn.model_selection import GridSearchCV

param_grid = {
    "classifier__n_estimators": [100, 200],
    "classifier__max_depth": [5, 10, None]
}

grid_search = GridSearchCV(
    pipeline, param_grid, scoring="f1", cv=5, n_jobs=-1
)

grid_search.fit(X_train, y_train)

```

Q17 — CNN in PyTorch

(a) CNN model class:

```

import torch
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # Input: (batch, 3, 64, 64)
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1) # -> (batch, 16, 64, 64)
        self.pool1 = nn.MaxPool2d(2) # -> (batch, 16, 32, 32)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1) # -> (batch, 32, 32, 32)
        self.pool2 = nn.MaxPool2d(2) # -> (batch, 32, 16, 16)
        self.relu = nn.ReLU()

        # Flattened size: 32 channels * 16 * 16 = 8192
        self.fc1 = nn.Linear(32 * 16 * 16, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool1(self.relu(self.conv1(x)))
        x = self.pool2(self.relu(self.conv2(x)))
        x = x.view(x.size(0), -1) # flatten
        x = self.relu(self.fc1(x))
        return self.fc2(x) # raw logits - no softmax here

```

Explain in the exam: with `padding=1`, `kernel=3`, convolution preserves spatial size; each `MaxPool2d(2)` halves it. Starting at $64 \times 64 \rightarrow$ after two pools $\rightarrow 16 \times 16$, with 32 channels $\rightarrow$ flatten to $32 \times 16 \times 16 = 8192$.

(b) Training loop for one epoch:

```

def train_one_epoch(model, train_loader, val_loader, optimizer, criterion, device):
    model.train()
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad() # clear old gradients from last step
        outputs = model(images) # forward pass
        loss = criterion(outputs, labels)
        loss.backward() # backward pass - computes gradients
        optimizer.step() # updates weights using those gradients

    model.eval()
    val_loss = 0.0
    with torch.no_grad(): # no gradient tracking needed for validation
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            val_loss += criterion(outputs, labels).item()
    return val_loss / len(val_loader)

```

- `zero_grad()` : PyTorch accumulates gradients by default, so you must clear them each step or they'd add up across batches.
- `loss.backward()` : computes gradients of the loss w.r.t. every parameter (backpropagation).
- `optimizer.step()` : actually updates the weights using those gradients.
- `torch.no_grad()` + `.eval()` : disables gradient tracking (saves memory/speed) and turns off training-only layers like Dropout/BatchNorm updates, since we're not learning during validation.

(c) Overfitting: This is **overfitting**. Three fixes: 1. **Dropout** — randomly zeroes some activations during training, forcing the network not to rely on any single path, improving generalization. 2. **Data augmentation** — random crops/flips/rotations effectively grow the training set, so the model sees more variation and memorizes less. 3. **Early stopping** — stop training once validation loss starts rising, keeping the model from continuing to fit noise in the training set.

Q18 — RAG System

(a) Architecture — two phases:

Ingestion: Load PDFs → extract text → **chunk** the text into smaller passages (with **overlap** between chunks so context isn't cut off mid-sentence/idea at chunk boundaries) → generate an embedding vector for each chunk → store (embedding + original text + metadata like source file/page) in a **vector database**.

Query time: User asks a question → embed the question with the same embedding model → **retrieve** the top-k most similar chunks from the vector DB (via cosine similarity) → construct a prompt containing the retrieved chunks as context → send to the LLM → return answer with citations back to the source chunks.

Embeddings are stored in a vector database (not a relational table) because similarity search requires fast **approximate nearest neighbor** lookups across high-dimensional vectors — something relational databases aren't built to do efficiently at scale.

(b) Minimal RAG code:

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

embedder = SentenceTransformer("all-MiniLM-L6-v2")

def chunk_text(text, chunk_size=500, overlap=50):
    chunks = []
    start = 0
    while start < len(text):
        chunks.append(text[start:start + chunk_size])
        start += chunk_size - overlap
    return chunks

# --- Ingestion ---
chunks = chunk_text(document_text)
embeddings = embedder.encode(chunks)
index = faiss.IndexFlatL2(embeddings.shape[1])
index.add(np.array(embeddings))

# --- Query time ---
def retrieve(query, k=3):
    query_vec = embedder.encode([query])
    distances, indices = index.search(np.array(query_vec), k)
    return [chunks[i] for i in indices[0]]

def build_prompt(query, retrieved_chunks):
    context = "\n\n".join(retrieved_chunks)
    system_prompt = (
        "You are a helpful assistant. Answer ONLY using the context provided below. "
        "If the answer is not in the context, say you don't know. Cite which excerpt you used."
    )
    return f"{system_prompt}\n\nContext:\n{context}\n\nQuestion: {query}"
```

(c) Causes of confident-but-wrong answers (retrieval-side) + fixes: 1. **Chunking too large/small or losing context** → tune chunk size and overlap so each chunk is self-contained. 2. **Irrelevant chunks retrieved** (poor embedding match, e.g. ambiguous wording) → improve retrieval with hybrid search (keyword + semantic) or re-ranking of top-k results. 3. **Outdated or duplicate documents in the vector store** → add document versioning/deduplication and refresh the index when source docs change.

RAG is the *wrong* choice when the need is a **fixed output style/format learned from many examples**, or teaching genuinely new reasoning/behavior — that's better solved with fine-tuning.

Q19 — Deployment (FastAPI + Docker)

(a) FastAPI app:

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib

app = FastAPI()
model = None

class PredictRequest(BaseModel):
    feature1: float
    feature2: float
    feature3: float

@app.on_event("startup")
def load_model():
    global model
    model = joblib.load("model.pkl")  # loaded once, not per request

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict")
def predict(request: PredictRequest):
    try:
        features = [[request.feature1, request.feature2, request.feature3]]
        prediction = model.predict(features)[0]
        confidence = max(model.predict_proba(features)[0])
        return {"prediction": prediction, "confidence": confidence}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

(b) Dockerfile:

```

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

python:3.11-slim keeps the image small. requirements.txt is copied and installed **before** the rest of the code because Docker caches layers — if only your app code changes (not dependencies), Docker reuses the cached install layer instead of reinstalling everything, speeding up rebuilds. Run with:

```

docker build -t my-model-api .
docker run -p 8000:8000 my-model-api

```

(c) Versioning, tracking, monitoring: Use **MLflow** to log each training run's parameters, metrics, and the resulting model artifact, tagging each with a version number. The API can accept a `model_version` parameter/route (e.g. `/predict/v3`) and load the corresponding registered model from the MLflow model registry.

In production, monitor: prediction latency, error rates, and **data drift** — a change in the statistical distribution of incoming feature data compared to training data (e.g., customer behavior shifts over time), which silently degrades model accuracy even though the model itself hasn't changed. Track drift with statistical tests (e.g., KS test) or drift-monitoring tools, alongside monitoring live accuracy/precision if ground-truth labels become available later.

Q20 — Agentic AI with n8n (support ticket assistant)

(a) Workflow design (ordered nodes):

1. **Webhook node** (trigger) — receives the ticket submission from the web form (customer message, ID, etc.)
2. **AI Agent node** — classifies urgency (e.g., low/medium/high/critical) from the ticket text
3. **IF/Switch node** — branches based on urgency:
 - If **critical** → **Slack node** (posts to a channel for a human)
 - Else → continue the automated path
4. **HTTP Request node** — calls the internal order-status API using the customer ID
5. **AI Agent node** — drafts a reply using the ticket text + order status as context
6. **Database node** (e.g., Postgres/MySQL node) — saves the ticket, classification, and drafted reply
7. **Email node** — sends the drafted reply to the customer
8. **Respond to Webhook node** — sends a confirmation response back to the web form

Data passed: the webhook payload flows forward and is progressively enriched — ticket text → + urgency label → + order status → + drafted reply — until it's logged and sent.

(b) Tool JSON schema + reasoning loop:

```
{
  "name": "get_order_status",
  "description": "Retrieves the current status of a customer's order given their customer ID.",
  "parameters": {
    "type": "object",
    "properties": {
      "customer_id": {
        "type": "string",
        "description": "The unique ID of the customer whose order status is needed."
      }
    }
  },
  "required": ["customer_id"]
}
```

Reasoning loop: the AI Agent reads the ticket, decides it needs order info to respond accurately, and **returns a structured tool call** (function name + `customer_id` argument) instead of a final answer. n8n executes the actual HTTP Request node with that argument, gets the real API response, and feeds the result back into the agent's context. The agent then reasons again — if it now has enough information to fully answer, it produces the final draft reply; if not, it can call another tool. It “decides it's finished” once its output no longer includes a tool call, just a final text response.

(c) Production hardening:

- **External API down** — add retry logic with exponential backoff in the HTTP Request node (n8n supports retry-on-fail), and a fallback path (e.g., queue the ticket or notify support) if retries are exhausted.
- **Duplicate webhook deliveries** — use an **idempotency key** (e.g., ticket ID) — check the database for that ID before processing again, so the same ticket isn't answered/emailed twice.
- **Securing webhook + credentials** — use a secret token/HMAC signature validation on the webhook endpoint, store API keys/credentials in n8n's built-in credential manager (never hard-coded in nodes), and restrict the webhook to HTTPS only.

Last-minute priority if time is short

If you only have 30 minutes: focus on **Q14, Q15, Q20** (agentic AI/n8n — this is clearly the role's core focus) and **Q4, Q6, Q13** (leakage, imbalance, RAG vs fine-tuning — these come up constantly in ML interviews). Being able to *talk through* Q20 clearly, even without perfect code, will matter most for this specific role.